AGENTIC AI IN BUSINESS

Coding & Agents

From Reliable Agents to Real-World Business Decision Systems

OUTLINE

Course Map

What can be reused if you want to build a reliable business agents in the future?

Vibe Coding	Coding Agents	Reliability	Read- world Design
From vibe coding to coding agents	Observable state	Verification	Formalization
	Actionability	Recovery policies	Example
		Permission boundaries	

OUTCOMES

Learning Outcomes

Decide when to use an agent

Assess clear objectives, observable state, verification and reversible actions.

Make the agent reliable

Choose feedback, recovery, constraints and escalation paths for a workflow.

Understand code as a meta-capability

Explain how code can extend an agent's capabilities in reasoning, constraint enforcement, system adaptation, and agent creation.

A strong design explains both the action and the evidence needed to trust it.

PART 1

Vibe Coding

What is Vibe Coding

@Anthropic
OpenAI founding member / Tesla AI Director

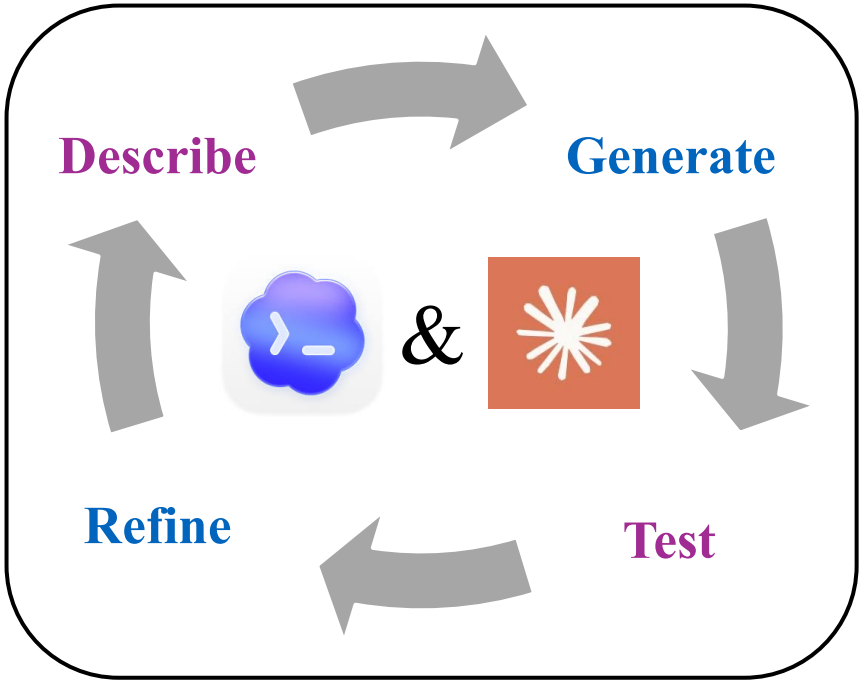


Andrej Karpathy @karpathy · February 3, 2025

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

1,453 6,065 33,000 7.11 million

Workflow



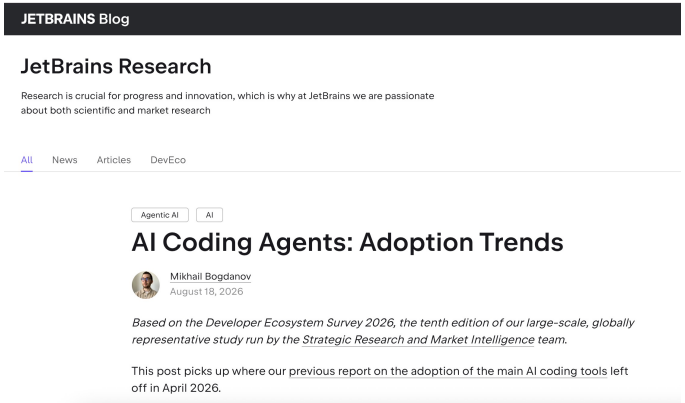
Technical details?

What do you want

How clearly you say

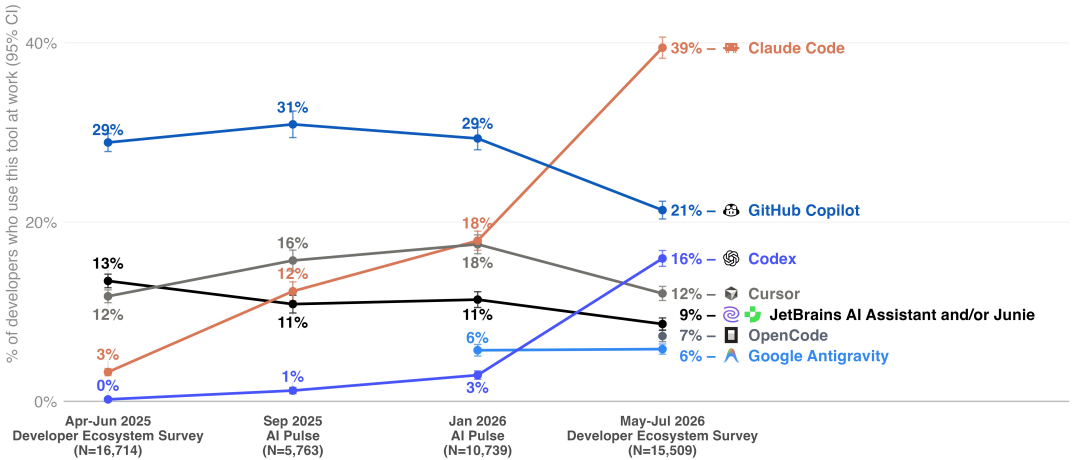
Agents Are Already Doing Real Work

90% of professional developers reported using AI coding agents at work at least *weekly*;
68% used them *daily*



surveyed in 2026

Use at work
Which of these AI tools do you use for your coding and other development-related tasks at work?
(Multiple answers were possible; only a subset of tools is shown)



From Vibe Coding to Coding Agents

Natural-language code generation becomes agentic

Vibe coding loop



Human-driven iteration

- User describes the desired requirements
- Model may inspect files and generate/edit code
- **User** decides what to try next
- **User** judges whether the result is acceptable

Agentic coding loop

- User describes the desired requirements
- Agent inspects files, edits code, and runs tools
- **Agent** decides the next action from feedback
- **Agent** continues or stops based on verification

The Minimal Coding-Agent Loop

ReAct-style loop, with strong execution feedback and verification work.

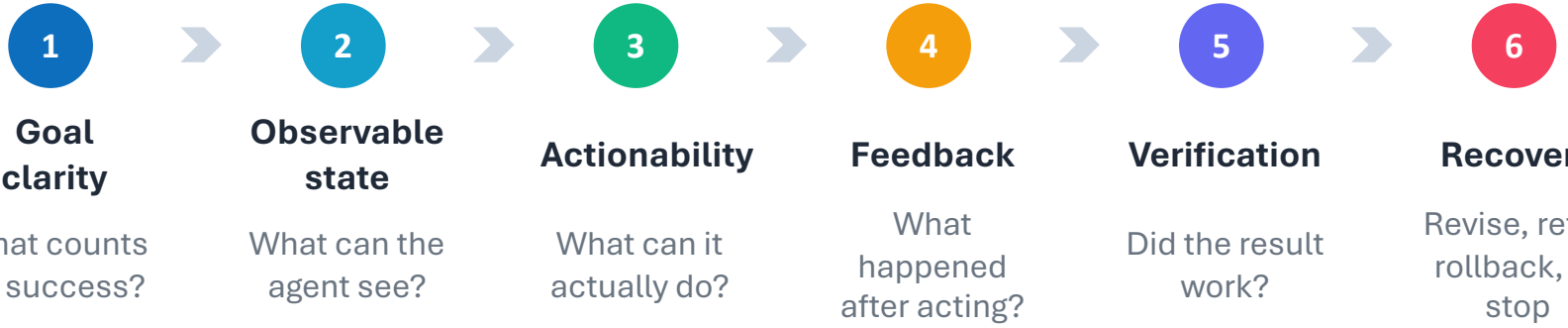


Vs ReAct

- Observation often comes from execution
- Often has executable verification
- Can stop when tests/criteria pass

AGENTIC CODING

Core Concepts



A loop becomes agentic when the system lets the agent see, act, check, and recover

PART 2

How Coding Agents Work

Coding provides an executable feedback loop

Goal Clarity

The system cannot verify progress if the target itself is underspecified

User request

“Generate a product recommendation list that users will like .”

❌ What you might get

Picks random products across unrelated categories (toys, office supplies, fashion)

- No clear format: some items have price, some don’t
- Uses generic descriptions like “This product is good”
- List length inconsistent

✅ What should you specify

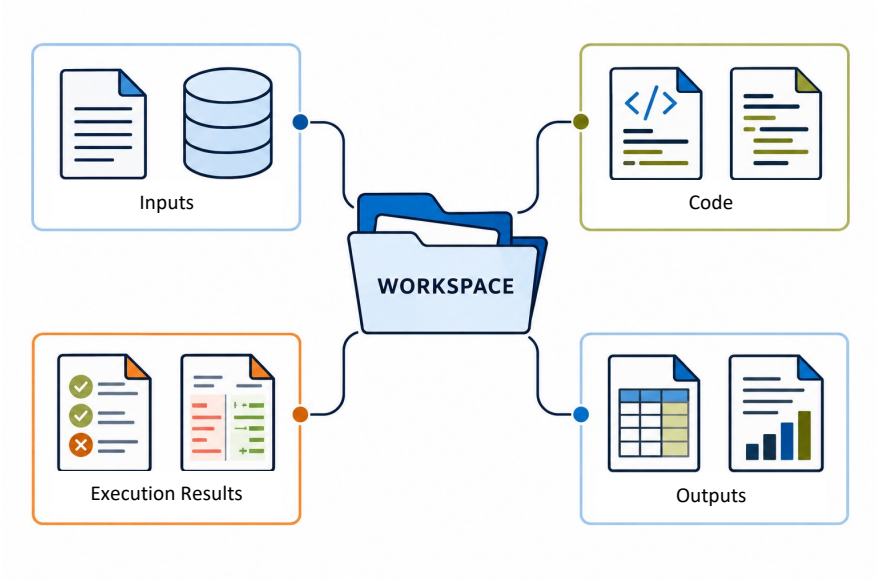
- What kind of products?
- How many items?
- Target user or use case?
- What info to include (price, brand, description)?

-Which aspect matters most?

.....

Observable State

The agent can inspect the current workspace, including inputs, code, outputs, and execution results



Current state is visible

Workspace = a persistent working environment where the agent can read, modify, and store task-related state

Changes are traceable

The agent can inspect what changed and what happened after each action

Actionability

The agent must be able to take real actions on the environment

Passive assistant

“You should change this function.”

- Advice remains outside the system
- A human must edit, run and check
- No environment state changes yet

turns advice into action

Actionable agent



The action changes the environment, so the next observation can inspect the new state.

Chatbot

AI Agents

CODING AGENT

Feedback

Structured observations reduce the amount the model has to guess

Weak feedback

“It failed.”

The next action has little diagnostic information.

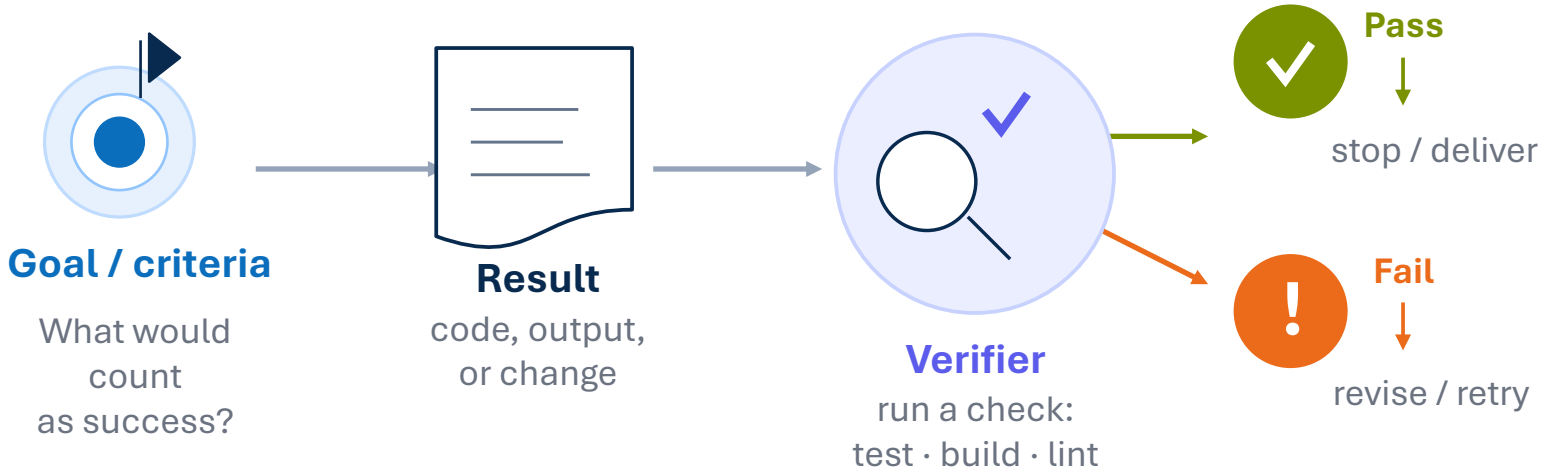
Structured feedback

```
FAIL  test_checkout.py:84
expected: 200
received: 403
```

Good feedback tells the agent what happened and where to look next

Verification

Structured observations reduce the amount the model has to guess



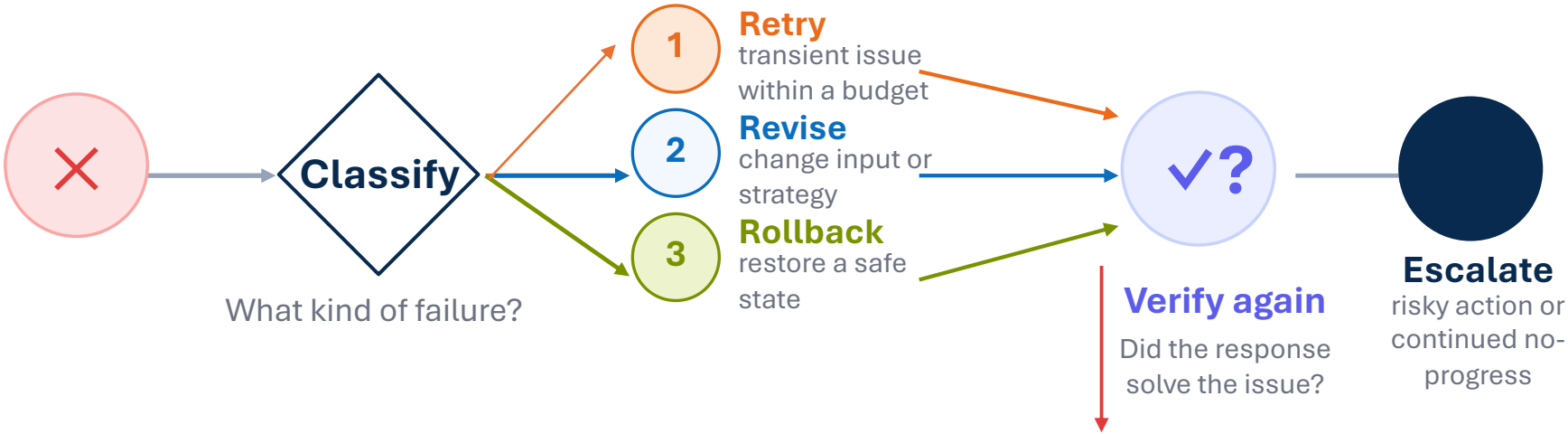
Clarity and Verification

Both the target and the feedback signal shape what an agent can do reliably

Requirements	Manual review	Automated verification
Vague goal	Hard to evaluate progress Human judgment carries the task	A signal exists The intended target is still unclear
Clear goal	The task is specified Verification remains slow	Executable acceptance tests The most coding-like setting

Recovery

When a check fails, the agent needs a recover within clear limits



CODING AGENT

Why Coding Is Agent-Friendly

Coding provides an environment where actions are observable, testable, and recoverable.

Property	Evidence available to the agent
Clear state	Files, diffs and logs
Executable actions	Commands and edits
Fast feedback	Tests and error messages
Rollback	Git history and isolated environments
Verification	Unit tests and review

CODING AGENT

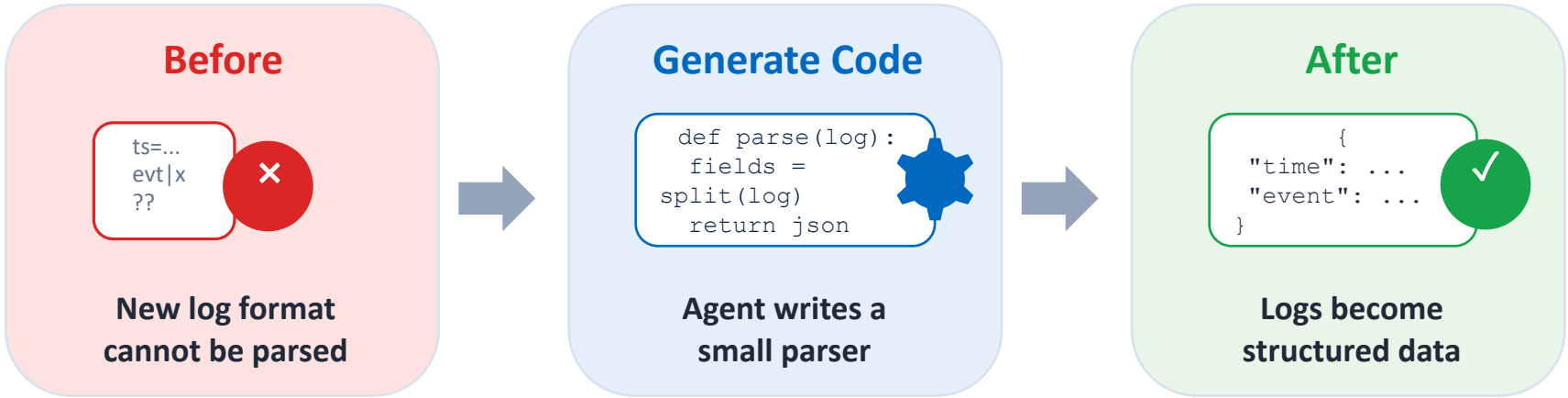
Code as a Meta-Capability

One more reason coding is special: Code does not only provide actions — it can create new capabilities.

Use	Compose	Create
Call a known tool	Compose tools into workflows	Create a new capability
Search a database or browse a page.	Fetch data, analyze it and build a report.	Write a parser, adapter or evaluator.
Fixed action space		

Open-ended analysis benefits from code. A narrow fixed workflow may need less flexibility.

Example : Adaptive Log Parsing



Meta-capability does not mean “any tool from nothing”

Human-defined primitives
read files · run code · API access · permissions

Agent-generated derived tools
parser · converter · API wrapper · validator

We still define the primitive tools and boundaries; code composes them into task-specific capabilities.

PART 3

What Makes Agents Reliable

What can we learn from coding agents that applies to agents more generally?

RELIABILITY

Four Reliability Levers

Feedback

Does the agent know what happened after the action ?

Verification

Can the agent verify that the outcome is correct or acceptable?

Recovery

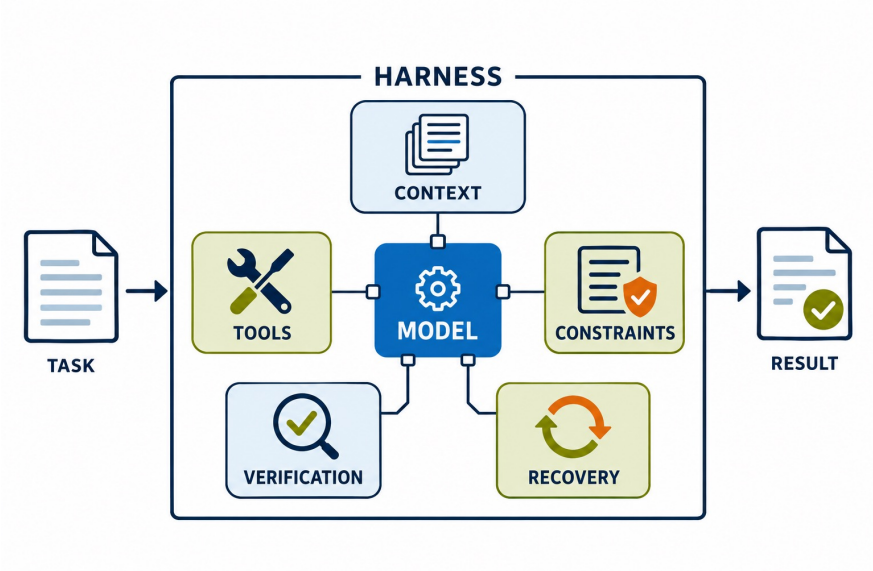
Can it retry, revise, roll back or escalate?

Constraints

Which rules must the system enforce?

The Model Is Part of a Larger System

The harness controls information, execution and feedback



Model

Interprets the task and proposes an action.

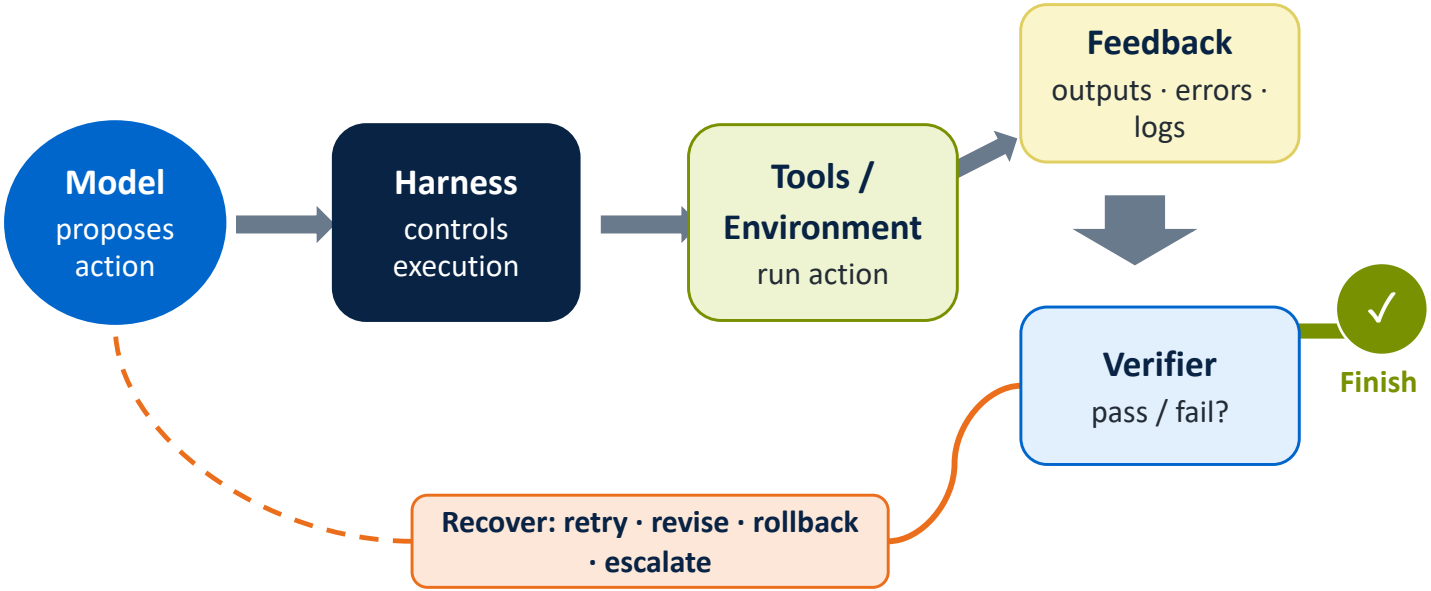
Harness

Supplies context and tools, checks actions, and manages recovery.

Agent performance depends on both the model and the system around it.

How the Harness Makes the Loop Reliable

It executes actions with limits, returns structured feedback, and decides whether to continue



1. **Route actions** to allowed tools
2. **Capture results** as feedback
3. **Run checks** before finishing
4. **Limit recovery** within clear limits

Valid Calls Can Still Make No Progress

Repeated execution does not guarantee new information

Repeated tool trace

```
01  read missing.txt   NOT FOUND
02  read missing.txt   NOT FOUND
03  read missing.txt   NOT FOUND
04  read missing.txt   NOT FOUND
```

Detection signals

Same tool and arguments
Same error fingerprint
No state change
Time or cost limit reached

Recovery

1 Classify failure
Which layer failed?

→

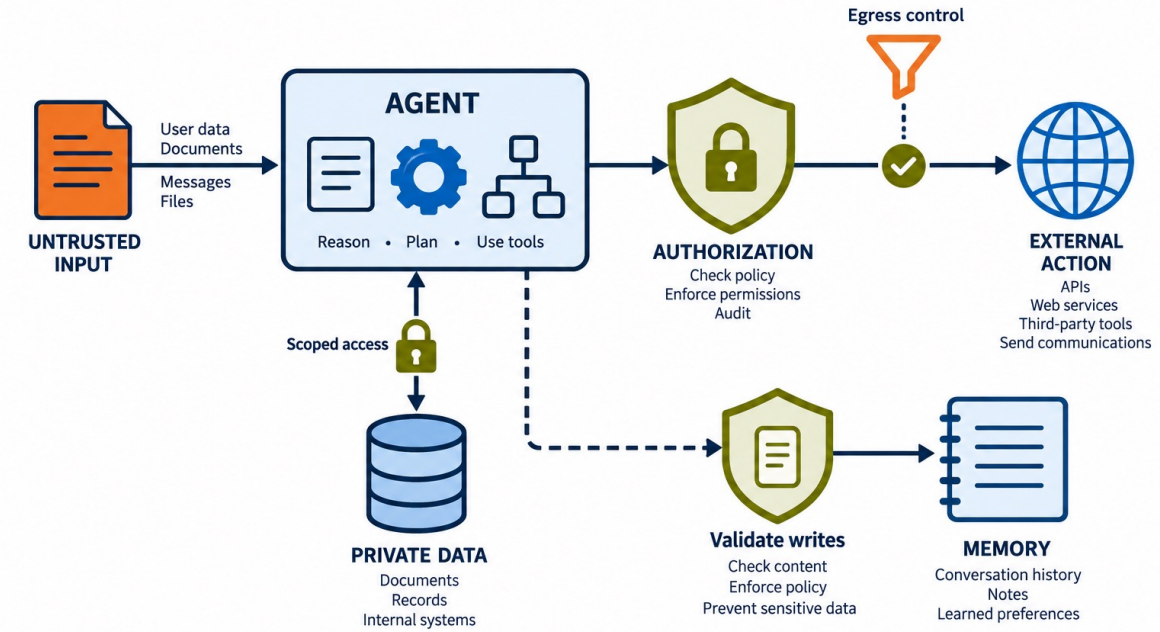
2 Choose response
Retry, correct, restore, or escalate

→

3 Apply rule
Budget, verify, or hand over

Layer	Typical failure	Recovery response	Stop when...
API	Rate limit or transient timeout	Retry with budget	attempt / time limit
Tool	Bad parameters or exceptions	Correct input or tool state	mismatch unresolved
Context	Missing result or overflow	Restore usable trajectory	state verified
Control	Repeated actions or no progress	Change strategy or escalate	new strategy / human help

Trust Boundaries



Access

Scoped credentials limit which private data the agent can read.

Action

Authorization and egress checks constrain effects outside the system.

SECURITY

Layered Constraint Design

Vibe-coding rules are useful guidance, but reliable agents need deterministic enforcement

1 Prompt rule

“Keep the code simple.”
“Do not add features beyond the request.”

Useful for behavior shaping, but the model can still ignore it.

2 Project instruction

AGENTS.md / CLAUDE.md
- test commands
- code style
- restricted folders

A stable project-level memory for the agent.

3 Enforcement layer

Permission checks
CI / lint / tests
Server-side validation
Approval gates

The system allows or rejects the action.

Example 1

How to build a business rule into an enforcement layer

1 Policy

What the rule is
Natural-language rules

Refunds are allowed only within 30 days of purchase.

2 Tool contract

How to request the action
Schema and checklist

refund_order(order_id, amount, reason)

3 Enforcement

Is the action allowed?
Server-side validation

Server checks the purchase date from the database.
≤ 30 days → allow
> 30 days → reject

A New Way of Setting the Boundary



Diogo Almeida

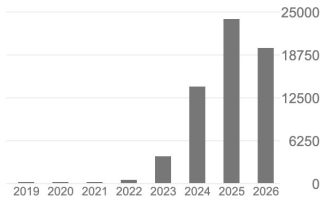
FOLLOW

ex-OpenAI
No verified email - Homepage
Deep Learning Large Language Models

TITLE	CITED BY	YEAR
Training language models to follow instructions with human feedback L Ouyang, J Wu, X Jiang, D Almeida, C Wainwright, P Mishkin, C Zhang, ... Advances in neural information processing systems 35, 27730-27744	31800	2022
Gpt-4 technical report J Achiam, S Adler, S Agarwal, L Ahmad, I Akkaya, FL Aleman, D Almeida, ... arXiv preprint arXiv:2303.08774	28144	2023
Resnet in resnet: Generalizing residual architectures S Targ, D Almeida, K Lyman arXiv preprint arXiv:1603.08029	1863	2016

Cited by

	All	Since 2021
Citations	63834	63160
h-index	17	16
i10-index	20	19



Jev

“A new type of evaluation to measure how well AI works within code.”

https://typesafe.ai/blog/introducing-system-one-models-and-jev

https://www.google.com/search?q=jev+demo&oq=jev+demo&gs_lcrp=EgZjaHJvbWUyBggAEEUYOTIICAEQABgWGB4yCAgCEAAYFhgeMg0IAxAAGIYDGIAEGIoFMg0IBBAAGIYDGIAEGIoF0gEIMjMxNGowajeoAgCwAgA&sourceid=chrome&source=chrome.ob&ie=UTF-8#fpstate=ive&vld=cid:4e1162e7,vid:QbYBRjOaGOo,st:0

PART 4

From Reliable Agents to Real-World Agents

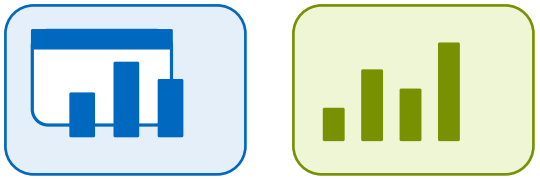
Can we bring the same reliability principles from coding agents into real-world decision systems?

Example 2

Verification must inspect the real output

1 Specify — 2 Generate — 3 Render — 4 Review — 5 Revise

revise until passes



Rendered artifact becomes inspectable evidence

Reviewer checks
- readability
- layout errors
- missing content
- visual consistency

……

Choose generation route

- Code route: precise, editable, reproducible
- Generative route: natural, rich, less controllable
- Mixed route: structure by code + style by model

Example 2

The generation route determines where precision lives: in explicit code or in the generative model.

CODE-BASED GENERATION

structure is explicit

Process Spec → Code → Run → Artifact

```
slide.add_text('Revenue')
chart = plot(data)
save('report.pptx')
```

Strengths Exact geometry · editable elements · reproducible data · deterministic constraints

Trade-off More engineering; visual style can feel rigid unless assets are designed separately.

GENERATIVE MODEL

structure is learned

Process Prompt → Model → Sample → Artifact

```
Prompt: 'Create a visual explaining
urban congestion with a clean,
editorial illustration style.'
```

Strengths Rich visual priors · natural composition · fast creation of open-ended imagery

Trade-off Harder to control exact layout, text, data values and editability; outputs may vary.

HYBRID

not a third route

Code: layout + text + data + **Model: illustration + style** → **Final editable artifact**

Use each route for what it controls best.

Choose code when exact structure matters; choose a generative model when visual richness matter.

Example 2

The file is not accepted when it is created. It is accepted when the rendered result passes review.

1 Generate → 2 Render → 3 Review → 4 Revise

repeat until evidence passes

EXECUTION CHECK

The program can succeed while the artifact still fails.

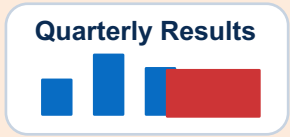
```
✓ pptx saved
✓ no runtime error
✓ file opens
```

What this proves

Only that the generation pipeline executed.

ARTIFACT CHECK

Rendering exposes failures that source code cannot reveal.



Visible failure

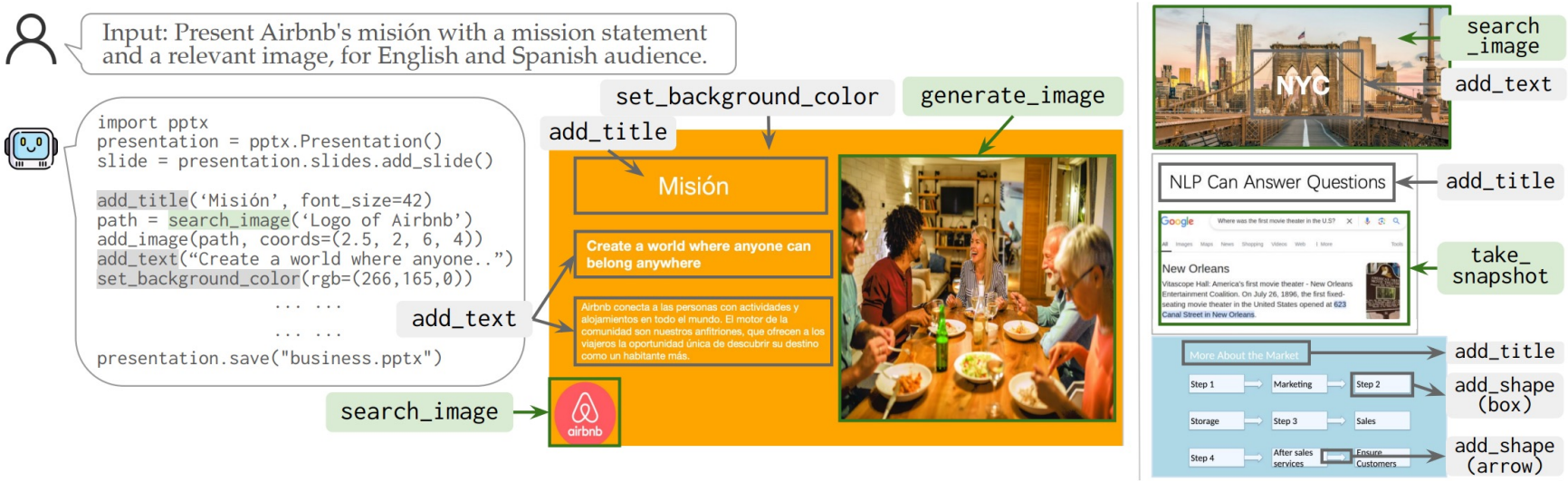
overlap · clipping · unreadable chart · missing content

Reviewer checks

CONTENT	LAYOUT	READABILITY	CONSISTENCY
missing / incorrect	clip / overlap	hierarchy / density	style / alignment

Feedback review findings **Verification** pass / fail judgment **Recovery** revise and render again

Example 3 - AUTOPRESENT



https://openaccess.thecvf.com/content/CVPR2025/papers/Ge_AutoPresent_Designing_Structured_Visuals_from_Scratch_CVPR_2025_paper.pdf

FORMALIZATION

Correct Execution ≠ Correct Modeling

Real decision
Reposition vehicles after a transit disruption.
What should improve?

Interpret / formalize
Choose variables, objective, constraints and assumptions.

Execute
Run the optimizer exactly.
Return a feasible plan.

Inspect result
Feasible ≠ useful.
Optimal ≠ appropriate.
Check the real goal.

Same solver, different formulation

Wrong model: minimize relocation distance → cheap movement, but long waiting time in the disrupted area

Better model: wait time + cost + coverage risk, with capacity, labor and approval constraints

Modeling errors hide in:

- **Objective** — wrong outcome
- **Constraint** — missing rule
- **Assumption** — bad forecast
- **Boundary** — needs approval

EXAMPLE

Urban Fleet Disruption Agent

The loop must handle uncertain demand and delayed business outcomes



Trigger: 7:30 AM subway disruption → demand hotspot near Zone C
Question: should the agent move vehicles automatically?

1 OBSERVE

State + forecast
fleet locations · traffic speed · depot capacity · demand forecast + uncertainty

2 PROPOSE

Candidate action
move k vehicles from A / B → C; choose routing, timing and operator assignment

3 CHECK

Before execution
hard constraints: budget, coverage floor, labor
simulation: waiting time, congestion, robustness
approval: high cost or unusual trade-off

4 ACT

Execute + monitor
staged rollout · KPI feedback · replanning if demand shifts or service drops

Summary

Reliable action depends on the structure of the whole decision system

1 **The agent includes more than the LLM**

The environment supplies tools, state and control.

2 **Reliability needs explicit mechanisms**

Verification, feedback, constraints and recovery work together.

3 **Tools perform deterministic work**

The model formalizes and orchestrates. Systems execute and check.

4 **Business agents need correct modeling**

Simulators, optimizers, policies and KPIs make action assessable.